МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ

Лабораторная работа № 1

по дисциплине «Математические основы баз данных»

по теме: «Проектирование и создание структуры базы данных»

Выполнил студент группы:
БВТ2505
Пондяков Игорь Дмитриевич
Проверил:
Быватов Семён Юрьевич

Москва 2026

Цель и задачи работы:

Цель: спроектировать и реализовать структуру реляционной базы данных для предметной области «Интернет-провайдер» в СУБД PostgreSQL, обеспечив её соответствие требованиям третьей нормальной формы (3НФ), ссылочной целостности и современным стандартам проектирования информационных систем.

Задачи:

1. Провести анализ предметной области и сформулировать перечень бизнес-правил, регламентирующих хранение и обработку данных.
2. Построить концептуальную и логическую ER-модель, включающую не менее 10 сущностей, связь «многие-ко-многим» (через ассоциативную таблицу) и иерархическую связь (самоссылку).
3. Выполнить нормализацию схемы данных до третьей нормальной формы (3НФ), продемонстрировав устранение аномалий вставки, обновления и удаления на конкретном примере.
4. Определить первичные ключи для каждой таблицы и обосновать выбор между суррогатными и естественными ключами.
5. Реализовать физическую модель базы данных средствами языка DDL PostgreSQL, задав корректные типы данных и ограничения целостности (PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK, DEFAULT, ON DELETE/ON UPDATE).
6. Составить словарь данных с описанием таблиц, столбцов, их типов и назначения.
7. Оформить все команды создания объектов единым воспроизводимым SQL-скриптом, позволяющим воссоздать схему с нуля.

Выполнение

1. Анализ предметной области

Описание предметной области:

Интернет-провайдер - это организация, предоставляющая физическим и юридическим лицам услуги доступа к сети Интернет, а также сопутствующие сервисы (цифровое телевидение, телефония, статический IP-адрес, антивирусная защита и т.д.). Деятельность провайдера охватывает полный жизненный цикл обслуживания клиента: от привлечения и подключения до технической поддержки и расчётов.

Основными участниками предметной области являются:

- Абоненты - физические лица, заключающие договор на оказание услуг связи;
- Тарифные планы - наборы параметров услуги (скорость, объём трафика, абонентская плата);
- Сотрудники - персонал провайдера (инженеры подключения, операторы техподдержки, менеджеры);
- Оборудование - технические средства (роутеры, модемы, ТВ-приставки), выдаваемые абоненту во временное пользование.

Ключевые бизнес-процессы, которые должна отражать база данных:

1. Подключение абонента - заключение договора, выбор тарифа, выдача оборудования, внесение первой оплаты.

2. Обслуживание договора - смена тарифного плана, подключение/отключение дополнительных услуг, приостановка и расторжение договора.
3. Расчёты - ежемесячное начисление абонентской платы, приём платежей, контроль задолженностей.
4. Техническая поддержка - приём заявок от абонентов, назначение исполнителей, фиксация результатов работ.
5. Управление ресурсами -учёт выданного оборудования, зон покрытия, каталога услуг.

Бизнес-правила:

На основе анализа предметной области сформулированы следующие бизнес-правила, которые определяют структуру проектируемой базы данных:

1. Абонент - это физическое лицо, идентифицируемое по паспортным данным. Один абонент может заключить несколько договоров в разное время (например, при переезде или после расторжения предыдущего), но в каждый момент времени активным может быть только один договор.
2. Тарифный план описывает базовые параметры услуги: скорость передачи данных (входящую и исходящую), абонентскую плату и тип подключения (оптика, Ethernet, ADSL). Каждый тариф имеет уникальное название.
3. Договор заключается между провайдером и абонентом на основании одного тарифного плана. Договор имеет статус (active - действующий, suspended - приостановлен, closed - закрыт) и сроки действия.
4. Дополнительные услуги (статический IP, антивирус, ТВ-пакеты) могут быть подключены к договору независимо от базового тарифа. Между

договорами и услугами существует связь «многие-ко-многим»: к одному договору можно подключить несколько услуг, и одна услуга может быть подключена к множеству договоров.

5. Услуги классифицируются по категориям, образующим иерархическую структуру (например: «Телевидение» -> «HD-каналы» -> «Спорт HD»). Корневые категории не имеют родителя; каждая подкатегория принадлежит ровно одной родительской категории.

6. Оборудование (роутер, модем, ТВ-приставка) выдаётся абоненту по договору. Каждая единица оборудования имеет уникальный серийный номер и может быть возвращена провайдеру при расторжении договора. К одному договору может быть привязано несколько единиц оборудования.

7. Платежи вносятся абонентом в счёт конкретного договора. По каждому платежу фиксируются сумма, дата и способ оплаты. История платежей хранится бессрочно и не может быть изменена задним числом.

8. Заявки в техническую поддержку создаются по договору и описывают проблему абонента. Каждая заявка имеет тип (подключение, поломка, консультация) и статус (new, in_progress, done). Заявка может быть назначена на конкретного сотрудника-исполнителя.

9. Сотрудник провайдера занимает определённую должность (инженер, оператор, менеджер) и может быть исполнителем по множеству заявок.

10. Абонентская плата начисляется ежемесячно согласно тарифному плану и подключённым дополнительным услугам. Задолженность возникает, если сумма платежей по договору меньше суммы начислений за период.

2. Проектирования ER-модели

На основе анализа предметной области и сформулированных бизнес-правил выделено 10 сущностей, которые войдут в проектируемую базу данных:

№	Сущность	Таблица	Описание
1	Абонент	subscribers	Физическое лицо - клиент провайдера. Хранятся персональные и контактные данные.
2	Тарифный план	tariffs	Описание базовой услуги: скорость, тип подключения, абонентская плата.
3	Договор	contracts	Юридическое соглашение между провайдером и абонентом. Фиксирует выбранный

			тариф, сроки и статус.
4	Категория услуги	service_categories	Классификатор дополнительных услуг с иерархической структурой (например, «Телевидение» -> «HD-каналы»).
5	Дополнительная услуга	services	Конкретная услуга (статический IP, антивирус, ТВ-пакет), которую можно подключить к договору.
6	Подключённая услуга	contract_services	Ассоциативная таблица, реализующая связь M:N между договорами и услугами.
7	Оборудование	equipment	Технические средства (роутер,

			модем, ТВ-приставка), выдаваемые абоненту по договору.
8	Платёж	payments	Факт оплаты по договору: сумма, дата, способ.
9	Заявка в поддержку	support_tickets	Обращение абонента в техническую поддержку с описанием проблемы.
10	Сотрудник	employees	Персонал провайдера, исполняющий заявки (инженеры, операторы).

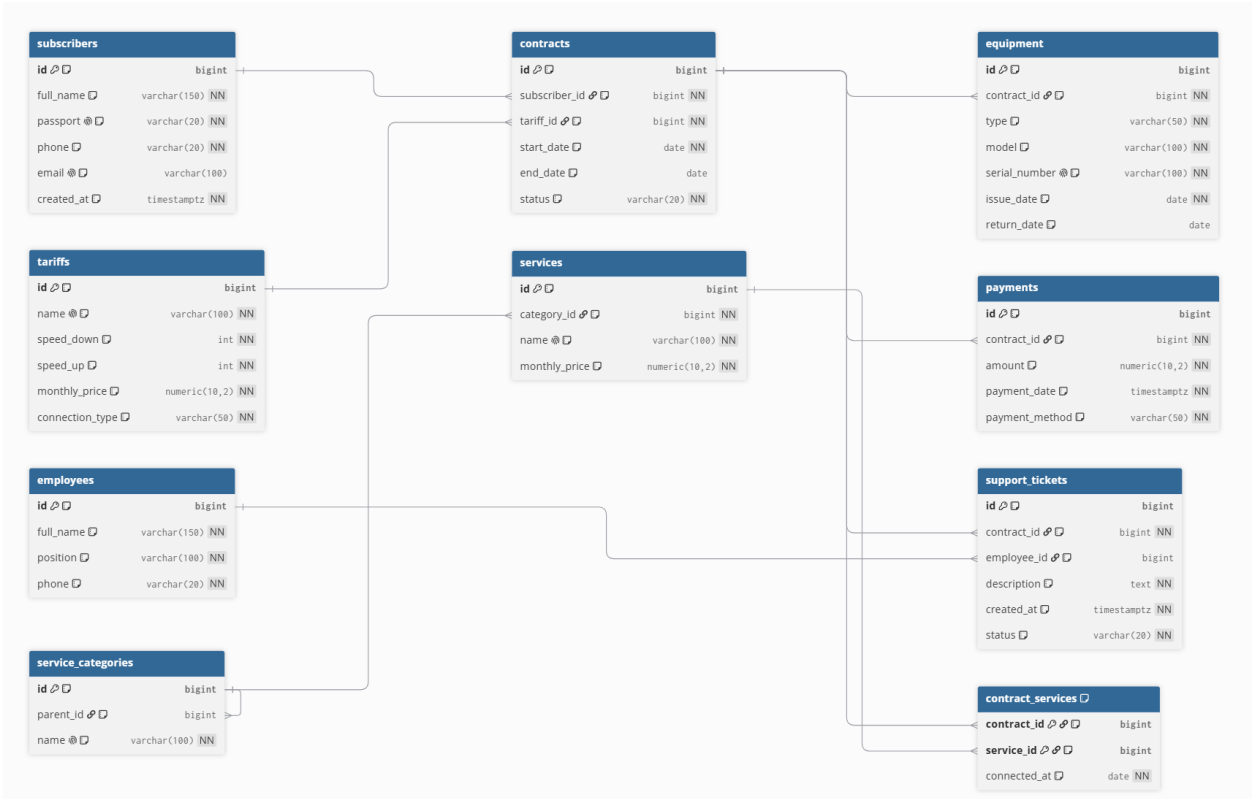
Связи между сущностями:

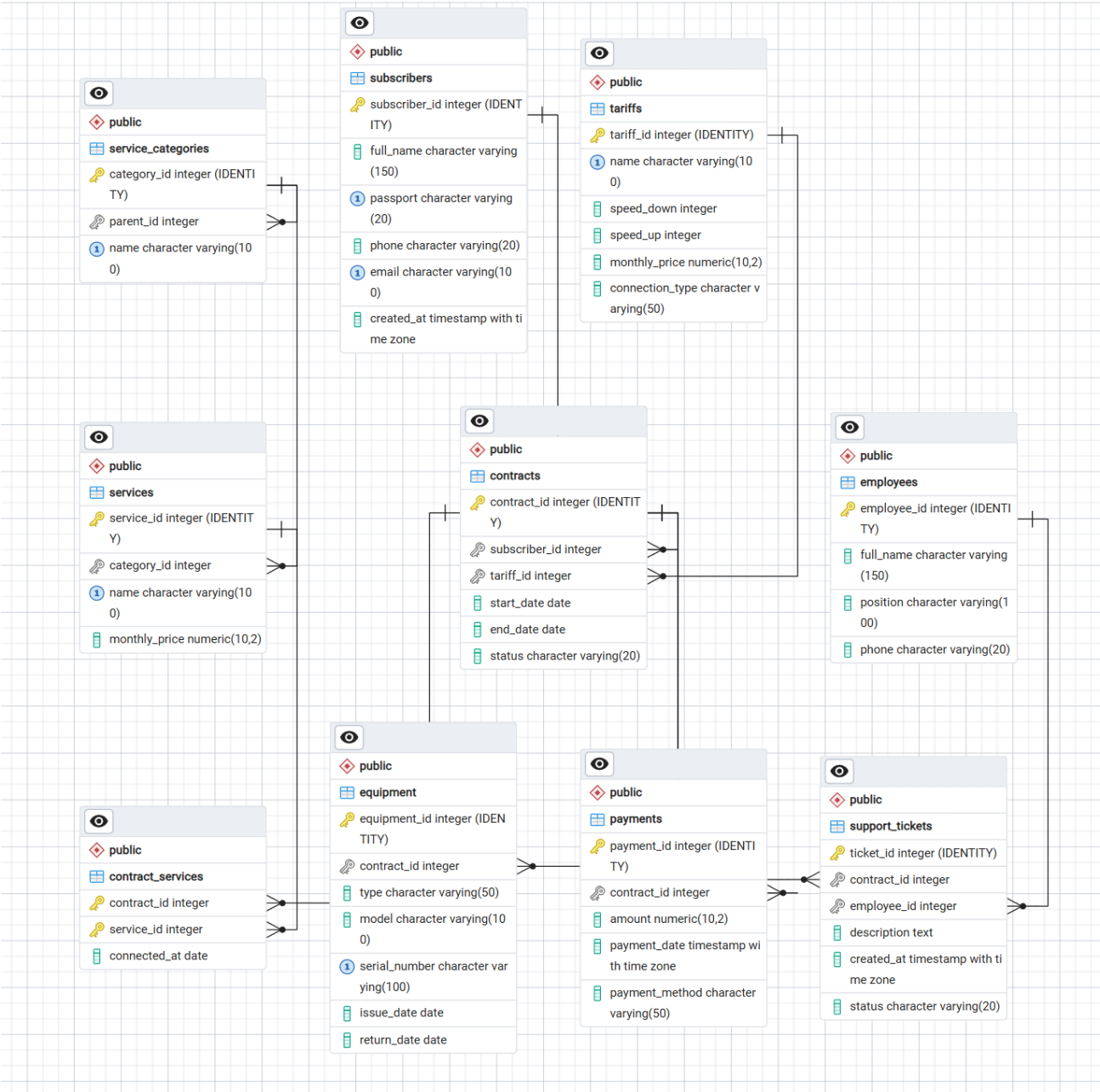
Связи между сущностями определяют, как данные соотносятся друг с другом. Для каждой связи указана кардинальность - сколько объектов одной сущности может соответствовать объекту другой.

№	Сущность 1	Связь	Сущность 2	Кардинальность	Тип Связи
1	Абонент	Заключает	Договор	1:N	Один абонент может иметь несколько договоров (исторически), но одновременно активен один.
2	Тариф	Используется в	Договор	1:N	Один тариф может фигурировать во множестве договоров.
3	Договор	Включает	Подключенная услуга	1:N	К одному договору можно подключить несколько услуг.
4	Услуга	Подключается к	Подключенная услуга	1:N	Одна услуга может быть подключена к множеству договоров.
5	Договор	Связан с	Услуга	M:N	Реализуется через ассоциативную таблицу contract_services.
6	Категория	Содержит	Категория	1:N	Иерархическая связь (самоссылка):

					подкатегория принадлежит одной родительской категории.
7	Категория	Классифицирует	Услуга	1:N	Каждая услуга относится к одной категории.
8	Договор	Использует	Оборудование	1:N	К одному договору может быть привязано несколько устройств.
9	Договор	Имеет	Платёж	1:N	По договору может быть множество платежей.
10	Договор	Создаёт	Заявка	1:N	По договору может быть подано несколько заявок.
11	Сотрудник	Исполняет	Заявка	1:N	Один сотрудник может быть исполнителем по многим заявкам.

ER-модель:


subscribers
id bigint
full_name varchar(150) NN
passport varchar(20) NN
phone varchar(20) NN
email varchar(100)
created_at timestamptz NN
contracts
id bigint
subscriber_id bigint NN
tariff_id bigint NN
start_date date NN
end_date date
status varchar(20) NN
equipment
id bigint
contract_id bigint NN
type varchar(50) NN
model varchar(100) NN
serial_number varchar(100) NN
issue_date date NN
return_date date
tariffs
id bigint
name varchar(100) NN
speed_down int NN
speed_up int NN
monthly_price numeric(10,2) NN
connection_type varchar(50) NN
services
id bigint
category_id bigint NN
name varchar(100) NN
monthly_price numeric(10,2) NN
payments
id bigint
contract_id bigint NN
amount numeric(10,2) NN
payment_date timestamptz NN
payment_method varchar(50) NN
employees
id bigint
full_name varchar(150) NN
position varchar(100) NN
phone varchar(20) NN
support_tickets
id bigint
contract_id bigint NN
employee_id bigint
description text NN
created_at timestamptz NN
status varchar(20) NN
service_categories
id bigint
parent_id bigint
name varchar(100) NN
contract_services
contract_id bigint
service_id bigint
connected_at date NN



4. Нормализация схемы

Нормализация - это процесс организации данных в базе данных, направленный на уменьшение избыточности и устранение аномалий при операциях вставки, обновления и удаления. Цель нормализации — привести схему к такой форме, при которой каждое неключевое поле зависит только от первичного ключа, от всего ключа целиком и ни от чего более.

На практике базовым целевым уровнем считается третья нормальная форма (3НФ), которая устраняет большинство аномалий и обеспечивает целостность данных без излишней фрагментации схемы.

Требования нормальных форм:

- 1НФ (первая нормальная форма): все значения в таблице атомарны (неделимы), нет повторяющихся групп и списков в одной ячейке.
- 2НФ (вторая нормальная форма): таблица находится в 1НФ, и каждый неключевой атрибут полностью зависит от всего первичного ключа (актуально для таблиц с составным ключом).
- 3НФ (третья нормальная форма): таблица находится в 2НФ, и ни один неключевой атрибут не зависит транзитивно от другого неключевого атрибута.

Рассмотрим гипотетическую ненормализованную таблицу Flat_Contracts, которая хранит всю информацию о договорах в одной плоской структуре:

contract_id	subscriber_name	subscriber_phone	tariff_name	tariff_price	equipment_serial	equipment_model
1	Иванов И.И.	+79001111111	Домашний 100	500	RT-001	TP-Link Archer
1	Иванов И.И.	+79001111111	Домашний 100	500	RT-001	MAG 250

2	Петров П.П.	+79002222222	Геймер 500	900	RT-002	ASUS RT-AC
3	Сидоров С.С.	+79003333333	Домашний 100	500	RT-003	TP-Link Archer

Аномалия обновления:

Если абонент Иванов И.И. меняет номер телефона с +79001111111 на +79009999999, необходимо обновить все строки, где фигурирует этот абонент. В нашем примере - две строки (contract_id = 1). Если забыть обновить хотя бы одну строку, данные рассогласуются: в базе будут храниться два разных номера телефона для одного абонента.

Проблема: атрибут subscriber_phone зависит от subscriber_name (или от идентификатора абонента), а не от первичного ключа таблицы (contract_id, equipment_serial). Это транзитивная зависимость, нарушающая 3НФ.

Аномалия вставки:

Мы не можем добавить в базу новый тарифный план "Студенческий 50" со стоимостью 300 рублей, пока хотя бы один абонент не заключит договор по этому тарифу. Причина: для вставки строки требуется значение contract_id (часть первичного ключа), а договора ещё не существует.

Проблема: информация о тарифе не может существовать независимо от договора, хотя логически тариф - самостоятельная сущность.

Аномалия удаления:

Если мы удаляем договор №2 (Петров П.П. расторг договор), мы безвозвратно теряем информацию о выданном ему оборудовании: серийный номер RT-002 и модель ASUS RT-AC. Если это оборудование нужно учесть при возврате или инвентаризации, данные уже недоступны.

Проблема: удаление договора приводит к потере информации об оборудовании, хотя эти сущности независимы.

Приведение к третьей нормальной форме:

Шаг 1: Создаём таблицу subscribers, которая хранит данные об абонентах

id	full_name	passport	phone	email
1	Иванов И.И.	1234 567890	+79001111111	ivanov@mail.ru
2	Петров П.П.	2345 678901	+79002222222	petrov@mail.ru
3	Сидоров С.С.	3456 789012	+79003333333	sidorov@mail.ru

Шаг 2: Выделение сущности "Тариф"

Создаём таблицу tariffs, которая хранит информацию о тарифных планах:

Id	name	speed_down	speed_up	monthly_price
1	Домашний 100	100	50	500
2	Геймер 500	500	250	900
3	Студенческий 50	50	25	300

Шаг 3: Выделение сущности "Договор"

Создаём таблицу contracts, которая связывает абонента с тарифом:

Id	subscriber_id	tariff_id	start_date	status
1	1	1	2024-01-15	active
2	2	2	2024-02-20	Active
3	3	1	2024-03-10	Active

Шаг 4: Выделение сущности "Оборудование"

Создаём таблицу equipment, которая хранит информацию о выданных устройствах:

id	contract_id	type	model	serial_number
1	1	Роутер	TP-Link Archer	RT-001
2	1	ТВ-приставка	MAG 250	TV-001
3	2	Роутер	ASUS RT-AC	RT-002
4	3	Роутер	TP-Link Archer	RT-003

Проверим на соответствие третьей нормальной форме таблицу Flat_Contracts, на примере которой демонстрировались аномалии, и ключевые таблицы, полученные после нормализации.

Исходная таблица Flat_Contracts (до нормализации) имела составной первичный ключ (contract_id, equipment_serial) и содержала атрибуты, зависящие не от всего ключа, а от его частей или друг от друга:

subscriber_name, subscriber_phone зависели только от contract_id (часть ключа) — нарушение 2НФ.

tariff_name, tariff_price также зависели только от contract_id — ещё одно нарушение 2НФ.

subscriber_phone зависел от subscriber_name транзитивно — нарушение 3НФ.

После декомпозиции на таблицы subscribers, tariffs, contracts, equipment:

subscribers (PK id): все неключевые атрибуты (full_name, passport, phone, email) зависят напрямую от id. Транзитивных зависимостей нет. 3НФ.

tariffs (PK id): атрибуты name, speed_down, speed_up, monthly_price зависят только от id. 3НФ.

contracts (PK id): атрибуты subscriber_id, tariff_id, start_date, status зависят только от id. 3НФ.

equipment (PK id): атрибуты contract_id, type, model, serial_number зависят только от id. 3НФ.

contract_services (составной PK (contract_id, service_id)): атрибут connected_at зависит от всего составного ключа целиком, а не от его части. 2НФ и 3НФ.

Остальные таблицы схемы (payments, support_tickets, services, service_categories, employees) спроектированы по тому же принципу: одна сущность — одна таблица, каждый неключевой атрибут зависит только от первичного ключа. Транзитивные зависимости отсутствуют во всей схеме.

5. Выбор и обоснование первичных ключей

Таблица	Первичный ключ	Тип ключа	Обоснование
subscribers	id	Суррогатный	Естественный кандидат - passport (паспорт). Он сохранён как UNIQUE, но не выбран как PK, так как паспортные данные чувствительны и теоретически могут быть изменены/исправлены
tariffs	id	Суррогатный	Естественный кандидат - name (название тарифа). Маркетинг может

			переименовать тариф («Домашний 100» → «Супер 100»), что при естественном ключе потребовало бы обновления всех договоров.
contracts	id	Суррогатный	Естественный кандидат - составной ключ (subscriber_id, start_date). Он громоздок для внешних ключей и не гарантирует уникальность при сложных бизнес-сценариях (например, приостановка и повторное подключение в один день).
service_categories	id	Суррогатный	Естественный кандидат - name. В иерархии названия могут повторяться в разных ветках (например, «HD» в

			телевидении и «HD» в интернете)
services	id	Суррогатный	Естественный кандидат - name. Сохранён как UNIQUE.
contract_services	(contract_id, service_id)	Естественный (составной)	Исключение из правила. Это ассоциативная таблица для связи M:N. Пара внешних ключей идеально описывает суть связи: услуга не может быть подключена к одному договору дважды. Введение суррогатного id здесь избыточно.
equipment	id	Суррогатный	Естественный кандидат - serial_number. Сохранён как UNIQUE. Использование его как PK рискованно

			из-за возможных опечаток при ручном вводе сотрудниками.
payments	id	Суррогатный	Естественного уникального атрибута нет (платежи - это просто поток транзакций). Суррогатный ключ единственно верное решение.
support_tickets	id	Суррогатный	Аналогично платежам, заявки не имеют стабильного естественного идентификатора.
employees	id	Суррогатный	Естественный кандидат - паспорт или СНИЛС. Использование персональных данных сотрудников в качестве PK нарушает принципы безопасности и удобства.

6. Единый SQL скрипт

```
DROP TABLE IF EXISTS support_tickets CASCADE;

DROP TABLE IF EXISTS payments CASCADE;

DROP TABLE IF EXISTS equipment CASCADE;

DROP TABLE IF EXISTS contract_services CASCADE;

DROP TABLE IF EXISTS contracts CASCADE;

DROP TABLE IF EXISTS services CASCADE;

DROP TABLE IF EXISTS service_categories CASCADE;

DROP TABLE IF EXISTS employees CASCADE;

DROP TABLE IF EXISTS tariffs CASCADE;

DROP TABLE IF EXISTS subscribers CASCADE;


CREATE TABLE service_categories (

   category_id   INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

   parent_id     INTEGER REFERENCES service_categories(category_id)

              ON DELETE RESTRICT ON UPDATE CASCADE,

   name          VARCHAR(100) NOT NULL UNIQUE

);


CREATE TABLE subscribers (

   subscriber_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

   full_name     VARCHAR(150) NOT NULL,

   passport      VARCHAR(20)  NOT NULL UNIQUE,
```

```
    phone         VARCHAR(20)  NOT NULL,
    email         VARCHAR(100) UNIQUE,
    created_at    TIMESTAMPTZ  NOT NULL DEFAULT now()
);

CREATE TABLE tariffs (
    tariff_id       INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name            VARCHAR(100)   NOT NULL UNIQUE,
    speed_down      INTEGER        NOT NULL CHECK (speed_down > 0),
    speed_up        INTEGER        NOT NULL CHECK (speed_up > 0),
    monthly_price   NUMERIC(10, 2) NOT NULL CHECK (monthly_price > 0),
    connection_type VARCHAR(50)    NOT NULL DEFAULT 'Оптика'
);

CREATE TABLE employees (
    employee_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    full_name   VARCHAR(150) NOT NULL,
    position    VARCHAR(100) NOT NULL,
    phone       VARCHAR(20)  NOT NULL
);

CREATE TABLE services (
    service_id    INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    category_id   INTEGER        NOT NULL REFERENCES service_categories(category_id)
```

```
                        ON DELETE RESTRICT ON UPDATE CASCADE,
    name          VARCHAR(100)   NOT NULL UNIQUE,
    monthly_price NUMERIC(10, 2) NOT NULL CHECK (monthly_price >= 0)
);


CREATE TABLE contracts (
    contract_id   INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    subscriber_id INTEGER      NOT NULL REFERENCES subscribers(subscriber_id)
                        ON DELETE RESTRICT ON UPDATE CASCADE,
    tariff_id     INTEGER      NOT NULL REFERENCES tariffs(tariff_id)
                        ON DELETE RESTRICT ON UPDATE CASCADE,
    start_date    DATE         NOT NULL DEFAULT CURRENT_DATE,
    end_date      DATE,
    status        VARCHAR(20)  NOT NULL DEFAULT 'active'
                        CHECK (status IN ('active', 'suspended', 'closed'))
);


CREATE TABLE contract_services (
    contract_id  INTEGER NOT NULL REFERENCES contracts(contract_id)
                    ON DELETE CASCADE ON UPDATE CASCADE,
    service_id   INTEGER NOT NULL REFERENCES services(service_id)
                    ON DELETE RESTRICT ON UPDATE CASCADE,
    connected_at DATE    NOT NULL DEFAULT CURRENT_DATE,
    PRIMARY KEY (contract_id, service_id)
```

```
);


CREATE TABLE equipment (
    equipment_id  INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    contract_id   INTEGER      NOT NULL REFERENCES contracts(contract_id)
                        ON DELETE RESTRICT ON UPDATE CASCADE,
    type          VARCHAR(50)  NOT NULL CHECK (type IN ('Роутер', 'ТВ-приставка', 'ONT-терминал')),
    model         VARCHAR(100) NOT NULL,
    serial_number VARCHAR(100) UNIQUE NOT NULL,
    issue_date    DATE         NOT NULL DEFAULT CURRENT_DATE,
    return_date   DATE
);


CREATE TABLE payments (
    payment_id     INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    contract_id    INTEGER        NOT NULL REFERENCES contracts(contract_id)
                          ON DELETE RESTRICT ON UPDATE CASCADE,
    amount         NUMERIC(10, 2) NOT NULL CHECK (amount > 0),
    payment_date   TIMESTAMPTZ    NOT NULL DEFAULT now(),
    payment_method VARCHAR(50)    NOT NULL DEFAULT 'Банковская карта'
);


CREATE TABLE support_tickets (
    ticket_id   INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```
    contract_id INTEGER      NOT NULL REFERENCES contracts(contract_id)
                        ON DELETE RESTRICT ON UPDATE CASCADE,
    employee_id INTEGER      REFERENCES employees(employee_id)
                        ON DELETE SET NULL ON UPDATE CASCADE,
    description TEXT         NOT NULL,
    created_at  TIMESTAMPTZ  NOT NULL DEFAULT now(),
    status      VARCHAR(20)  NOT NULL DEFAULT 'new'
                        CHECK (status IN ('new', 'in_progress', 'done'))
);
```

7. Словарь данных

Таблица 1. service_categories — Категории услуг

Столбец	Тип данных	Ограничения	Назначение
category_id	INTEGER	PK, IDENTITY, NOT NULL	Уникальный идентификатор категории
parent_id	INTEGER	FK → service_categories(category_id), ON DELETE RESTRICT	Ссылка на родительскую категорию (NULL для корневых)
name	VARCHAR(100)	NOT NULL, UNIQUE	Название категории

Таблица 2. subscribers — Абоненты

Столбец	Тип данных	Ограничения	Назначение
subscriber_id	INTEGER	PK, IDENTITY, NOT NULL	Уникальный идентификатор абонента
full_name	VARCHAR(150)	NOT NULL	ФИО клиента

passport	VARCHAR(20)	NOT NULL, UNIQUE	Паспортные данные
phone	VARCHAR(20)	NOT NULL	Контактный телефон
email	VARCHAR(100)	UNIQUE	Адрес электронной почты
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Дата и время регистрации

Таблица 3. tariffs — Тарифные планы

Столбец	Тип данных	Ограничения	Назначение
tariff_id	INTEGER	PK, IDENTITY, NOT NULL	Уникальный идентификатор тарифа
name	VARCHAR(100)	NOT NULL, UNIQUE	Название тарифного плана
speed_down	INTEGER	NOT NULL, CHECK (> 0)	Входящая скорость (Мбит/с)
speed_up	INTEGER	NOT NULL, CHECK (> 0)	Исходящая скорость (Мбит/с)
monthly_price	NUMERIC(10,2)	NOT NULL, CHECK (> 0)	Абонентская плата (руб.)
connection_type	VARCHAR(50)	NOT NULL, DEFAULT 'Оптика'	Тип подключения

Таблица 4. employees — Сотрудники

Столбец	Тип данных	Ограничения	Назначение
employee_id	INTEGER	PK, IDENTITY, NOT NULL	Уникальный идентификатор сотрудника
full_name	VARCHAR(150)	NOT NULL	ФИО сотрудника
position	VARCHAR(100)	NOT NULL	Занимаемая должность
phone	VARCHAR(20)	NOT NULL	Рабочий телефон

Таблица 5. services — Дополнительные услуги

Столбец	Тип данных	Ограничения	Назначение

service_id	INTEGER	PK, IDENTITY, NOT NULL	Уникальный идентификатор услуги
category_id	INTEGER	FK → service_categories(category_id), NOT NULL	Категория услуги
name	VARCHAR(100)	NOT NULL, UNIQUE	Название услуги
monthly_price	NUMERIC(10,2)	NOT NULL, CHECK (>= 0)	Стоимость услуги (руб.)

Таблица 6. contracts — Договоры

Столбец	Тип данных	Ограничения	Назначение
contract_id	INTEGER	PK, IDENTITY, NOT NULL	Уникальный номер договора
subscriber_id	INTEGER	FK → subscribers(subscriber_id), NOT NULL	Абонент-владелец договора
tariff_id	INTEGER	FK → tariffs(tariff_id), NOT NULL	Тарифный план договора
start_date	DATE	NOT NULL, DEFAULT CURRENT_DATE	Дата начала действия
end_date	DATE	—	Дата окончания (NULL, если активен)
status	VARCHAR(20)	NOT NULL, DEFAULT 'active', CHECK (IN 'active','suspended','closed')	Статус договора

Таблица 7. contract_services — Подключённые услуги (ассоциативная)

Столбец	Тип данных	Ограничения	Назначение
contract_id	INTEGER	PK, FK → contracts(contract_id), NOT NULL	Идентификатор договора
service_id	INTEGER	PK, FK → services(service_id), NOT NULL	Идентификатор услуги

connected_at	DATE	NOT NULL, DEFAULT CURRENT_DATE	Дата подключения услуги

Таблица 8. equipment — Оборудование

Столбец	Тип данных	Ограничения	Назначение
equipment_id	INTEGER	PK, IDENTITY, NOT NULL	Уникальный идентификатор оборудования
contract_id	INTEGER	FK → contracts(contract_id), NOT NULL	Договор, по которому выдано
type	VARCHAR(50)	NOT NULL, CHECK (IN 'Роутер','ТВ-приставка','ONT-терминал')	Тип устройства
model	VARCHAR(100)	NOT NULL	Модель устройства
serial_number	VARCHAR(100)	NOT NULL, UNIQUE	Заводской серийный номер
issue_date	DATE	NOT NULL, DEFAULT CURRENT_DATE	Дата выдачи
return_date	DATE	—	Дата возврата провайдеру

Таблица 9. payments — Платежи

Столбец	Тип данных	Ограничения	Назначение
payment_id	INTEGER	PK, IDENTITY, NOT NULL	Уникальный идентификатор платежа
contract_id	INTEGER	FK → contracts(contract_id), NOT NULL	Договор, по которому внесён платёж
amount	NUMERIC(10,2)	NOT NULL, CHECK (> 0)	Сумма платежа (руб.)
payment_date	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Дата и время оплаты
payment_method	VARCHAR(50)	NOT NULL, DEFAULT 'Банковская карта'	Способ оплаты

Таблица 10. support_tickets — Заявки в поддержку

Столбец	Тип данных	Ограничения	Назначение
ticket_id	INTEGER	PK, IDENTITY, NOT NULL	Уникальный номер заявки
contract_id	INTEGER	FK → contracts(contract_id), NOT NULL	Договор, по которому создана заявка
employee_id	INTEGER	FK → employees(employee_id), ON DELETE SET NULL	Сотрудник-исполнитель
description	TEXT	NOT NULL	Описание проблемы или запроса
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Дата и время создания
status	VARCHAR(20)	NOT NULL, DEFAULT 'new', CHECK (IN 'new','in_progress','done')	Статус обработки заявки

8. Вывод

В ходе выполнения лабораторной работы №1 была спроектирована и реализована структура реляционной базы данных для предметной области «Интернет-провайдер» в СУБД PostgreSQL 18.

В результате работы:

- Проведён анализ предметной области и сформулировано 10 бизнес-правил, определяющих логику хранения и обработки данных.
- Построена ER-модель, включающая 10 сущностей, связь «многие-ко-многим» (реализованную через ассоциативную таблицу contract_services) и иерархическую связь (самоссылку в таблице service_categories).
- Схема приведена к третьей нормальной форме (3НФ), что устраняет аномалии вставки, обновления и удаления данных.

- Для всех таблиц обоснован выбор суррогатных первичных ключей (INTEGER GENERATED ALWAYS AS IDENTITY), обеспечивающих стабильность и производительность, при сохранении естественных уникальных ограничений (UNIQUE) для бизнес-атрибутов.
- Физическая модель реализована средствами DDL с применением ограничений целостности (NOT NULL, UNIQUE, CHECK, DEFAULT, FOREIGN KEY с правилами ON DELETE/ON UPDATE).
- Составлен полный словарь данных и единый воспроизводимый SQL-скрипт, позволяющий воссоздать схему базы данных с нуля.